1

A declarative programming language is used to represent the following knowledge base.

```
01 person(jordan).
02 person(gita).
03 person(paolo).
04 person(cassie).
05 animal(cat).
06 animal(hamster).
07 animal(gecko).
08 animal(fish).
09 has_pet(cassie, gecko).
10 has_pet(paolo, fish).
11 has_pet(cassie, cat).
12 has_pet(jordan, cat).
```

These clauses have the following meanings:

Clause	Meaning
01	Jordan is a person
05	A cat is an animal
09	Cassie has a pet gecko

(a) Clive is a person who has a pet guinea pig and a pet gecko.

Write additional clauses to represent this information.

13 .....

14 .....

15 .....

16 .....

[4]

(b) Using the variable `PetOwner`, the goal:

```
has_pet(PetOwner, cat)
```

returns:

```
PetOwner = cassie, jordan
```

Write the result that is returned by the goal:

```
has_pet(cassie, PetAnimals)
```

`PetAnimals =` .....

..... [1]

(c) `Z` wants a pet `Y`, if `Z` is a person and `Y` is an animal and `Z` does not have a pet `Y`.

Write this as a rule.

`wants_pet(` ..... `,` ..... `)`

`IF` .....

.....

..... [5]

2

The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	**Operand**	
LDM	#n	Immediate addressing. Load the number n to ACC.
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store the contents of ACC at the given address.
STX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents from ACC to this calculated address.
ADD	<address>	Add the contents of the given address to the ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
JMP	<address>	Jump to the given address.
CMP	<address>	Compare the contents of ACC with the contents of <address>.
CMP	#n	Compare the contents of ACC with number n.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
LSL	#n	Bits in ACC are shifted n places to the left. Zeros are introduced on the right hand end.
LSR	#n	Bits in ACC are shifted n places to the right. Zeros are introduced on the left hand end.
IN		Key in a character and store its ASCII value in ACC.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

(a) A program stores a letter. The user is allowed nine attempts to guess the stored letter. The program outputs "?" and the user guesses a letter. If the user guesses the letter, the program outputs "*".

The following is pseudocode for this program.

```
REPEAT
   OUTPUT '?'
   INPUT GUESS
   IF GUESS = LETTERTOGUESS
      THEN
        OUTPUT '*'
        BREAK
   ELSE
      ATTEMPTS ← ATTEMPTS + 1
   ENDIF
UNTIL ATTEMPTS = 9
```

Write this program. Use the op codes from the instruction set provided.

Label	Op code	Operand	Comment
START:	LDM	#63	// load ASCII value for '?'
	OUT		// OUTPUT '?'
	IN		// input GUESS
	CMP	LETTERTOGUESS	// compare with stored letter
	JPE	GUESSED	// if correct guess, go to GUESSED
	LDD	ATTEMPTS	// increment ATTEMPTS
	INC	ACC	
	STO	ATTEMPTS	
	CMP	#9	// is ATTEMPTS = 9 ?
	JPE	ENDP	// if out of guesses, go to ENDP
	JMP	START	// go back to beginning of loop
GUESSED:	LDM	#42	// load ASCII for '*'
	OUT		// OUTPUT '*'
ENDP:	END		// end program
ATTEMPTS:	0		
LETTERTOGUESS:	'a'		

[11]

(b) Five numbers are stored, starting in the location labelled `NUMBERS`. A program is needed to multiply each of the numbers by 4 and store them back in their original location.

Write this program. Use the op codes from the instruction set on the opposite page.

Label	Op code	Operand	Comment
START:			// initialise the Index Register
			// load the value from NUMBERS
			// multiply by 4
			// store the new value in NUMBERS
			// increment the Index Register
			// increment COUNT
			// is COUNT = 5 ?
			// repeat for next number
ENDP:	END		
COUNT:	0		
NUMBERS:	22		
	13		
	5		
	46		
	12		

[10]